Μ Π Σ (FEM)
Ε 3D Σ Δ
Τ Κ Α Υ

Ρ Π
Α Μ : 6931
Μ : Μ Π Σ

Δ 2025

Π ΄

Α Π Λ Κ **18**

Β Ε Λ **18**

1 E

H M Π Σ (FEM) . T :

- **Π** (`preprocessor.py`): Δ ,
- **E** (`solver.py`): Σ FEM
- **M** (`postprocessor.py`): O

H 6 (DOF) : (U_x, U_y, U_z) (R_x, R_y, R_z).

1.1 K X

- ϒ
- A
- Γ ( )
- I
- O
- Δ Plotly
- A ,

2 M Θ

2.1 Δ T Σ Δ

Γ 6 DOF , 12×12. T :

- **A** : Σ E A
- **Σ** : Σ G J
- **K** : Σ E I_y, I_z

O :

$$k = \frac{EA}{L} \tag{1}$$

$$k = \frac{GJ}{L} \tag{2}$$

$$k_{,y} = \frac{12EI_y}{L^3}, \quad \frac{6EI_y}{L^2}, \quad \frac{4EI_y}{L}, \quad \frac{2EI_y}{L} \tag{3}$$

$$k_{,z} = \frac{12EI_z}{L^3}, \quad \frac{6EI_z}{L^2}, \quad \frac{4EI_z}{L}, \quad \frac{2EI_z}{L} \tag{4}$$

2.2 M Σ

O :

$$\mathbf{K} = \mathbf{T}^T \mathbf{K} \mathbf{T} \tag{5}$$

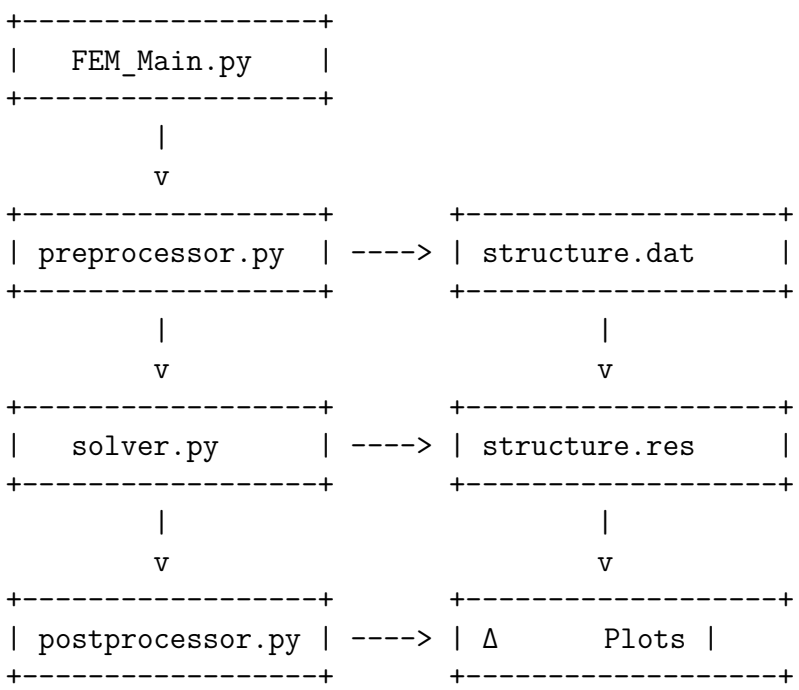
T 12×12 3×3.

3 A Λ

3.1 E M

T Python:

1. **FEM_Main.py**: K
2. **preprocessor.py**: Δ
3. **solver.py**: ϒ FEM
4. **postprocessor.py**: O
5. **geometry_utils.py**: B



Σ´ 1: P

4 M Π

O (preprocessor.py) ,
.

4.1 Δ K

H FEMPreProcessor:

```python
class FEMPreProcessor:
    """Handles geometry creation and boundary conditions for FEM analysis
    """

    def __init__(self):
        self.points = pd.DataFrame(columns=['Node Number', 'X', 'Y', 'Z'])
        self.elements = pd.DataFrame(columns=['Element Number', 'Node1', '
    Node2', 'E', 'V'])
        self.boundary_conditions = {}  # node_id: {'Ux': 0/1, 'Uy': 0/1, '
    Uz': 0/1, 'Rx': 0/1, 'Ry': 0/1, 'Rz': 0/1}
        self.loads = {}  # node_id: {'Fx': value, 'Fy': value, 'Fz': value,
     'Mx': value, 'My': value, 'Mz': value}
        self.materials = {'STEEL': {'E': 210e9, 'nu': 0.3}}

        # Beam properties (circular cross-section)
        self.diameter = 0.020  # 20mm diameter circular cross-section
```

Listing 1: A K FEMPreProcessor

4.2 I K Δ

Γ , :

```python
def _calculate_circular_properties(self, diameter):
    """Calculate geometric properties for circular cross-section"""
    r = diameter / 2.0

    # Cross-sectional area
    A = np.pi * r**2

    # Second moment of area (Iz = Iy for circular section)
    I = (np.pi * r**4) / 4.0
    Iz = I
    Iy = I

    # Torsional constant (polar moment of inertia)
    J = (np.pi * r**4) / 2.0

    return {'A': A, 'Iz': Iz, 'Iy': Iy, 'J': J, 'diameter': diameter, '
    radius': r}
```

Listing 2: ϒ I K Δ

A :

$$A = \pi r^2 \tag{6}$$

$$I_y = I_z = \frac{\pi r^4}{4} \tag{7}$$

$$J = \frac{\pi r^4}{2} = 2I \tag{8}$$

4.3 Δ Г

Н , ,
.

```
def _define_geometry_parameters(self):
    """Define geometry-specific parameters (not saved to .dat file)"""
    # COMPLEX GEOMETRY: Original crane/truss structure
    # Geometry parameters for this specific crane/truss structure
    self.L = 1.5 * 1.13      # Element length (m)
    self.A = 1.2 * 1.69      # Base dimension (m)
    self.B = 1.93            # Additional parameter B (m)
    self.phi = 60 + 9.1      # Rotation angle (degrees)
    self.A_0 = 6 * (0.5 + 0.13)  # Base cross-section parameter (
    dimensionless)
```

Listing 3: О П Г

4.4 Δ Σ

Т :

```
def _create_axis_aligned_elements(self, element_counter, tolerance,
    max_length):
    """Create elements parallel to X, Y, or Z axes"""
    axis_aligned_count = 0
    for i in range(len(self.points)):
        node1 = self.points.iloc[i]
        node1_num = int(node1['Node Number'])

        for j in range(i + 1, len(self.points)):
            node2 = self.points.iloc[j]
            node2_num = int(node2['Node Number'])

            dx = abs(node2['X'] - node1['X'])
            dy = abs(node2['Y'] - node1['Y'])
            dz = abs(node2['Z'] - node1['Z'])

            element_length = np.sqrt(dx**2 + dy**2 + dz**2)

            if element_length > max_length:
                continue

            # Check axis alignment
            is_x_aligned = dx > tolerance and dy < tolerance and dz <
    tolerance
            is_y_aligned = dx < tolerance and dy > tolerance and dz <
    tolerance
            is_z_aligned = dx < tolerance and dy < tolerance and dz >
    tolerance
```

Listing 4: Δ Σ П Ѧ

4.5 О Σ Ф

О :

```python
def add_boundary_condition(self, node_id, ux=1, uy=1, uz=1, rx=1, ry=1, rz=1):
    """
    Add boundary condition for a node

    Parameters:
    -----------
    node_id : int
        Node number (1-based)
    ux, uy, uz : int
        Translation constraints (1=fixed, 0=free)
    rx, ry, rz : int
        Rotation constraints (1=fixed, 0=free)
    """
    self.boundary_conditions[node_id] = {
        'Ux': ux, 'Uy': uy, 'Uz': uz,
        'Rx': rx, 'Ry': ry, 'Rz': rz
    }
```

Listing 5: Π O Σ

```python
def add_load(self, node_id, fx=0.0, fy=0.0, fz=0.0, mx=0.0, my=0.0, mz=0.0):
    """
    Add load (forces and moments) to a node

    Parameters:
    -----------
    node_id : int
        Node number (1-based)
    fx, fy, fz : float
        Force components in Newtons
    mx, my, mz : float
        Moment components in Newton-meters
    """
    self.loads[node_id] = {
        'Fx': fx, 'Fy': fy, 'Fz': fz,
        'Mx': mx, 'My': my, 'Mz': mz
    }
```

Listing 6: Π Φ

4.6 E Δ

O :

```python
def export_to_file(self, filename='data/structure.dat'):
    """Export structure to text file for solver"""
    print("\nEXPORTING STRUCTURE DATA")
    print("="*80)

    # Ensure directory exists
    os.makedirs(os.path.dirname(filename), exist_ok=True)

    with open(filename, 'w') as f:
        # Write header
        f.write("# FEM Structure Data File\n")
```

```
        f.write("# Generated by FEM Pre-Processor (Beam Elements)\n")
        f.write("# Author: Rakovalis Pavlos 6931\n\n")

        # Write parameters
        f.write("PARAMETERS\n")
        props = self._calculate_circular_properties(self.diameter)
        f.write(f"{self.E} {props['A']} {props['Iz']} {props['Iy']} {props['J']} {self.G} {self.diameter}\n\n")

        # Write materials
        f.write("MATERIALS\n")
        for mat_name, props in self.materials.items():
            f.write(f"{mat_name} {props['E']} {props['nu']}\n")
        f.write("\n")

        # Write nodes
        f.write("NODES\n")
        for idx in range(len(self.points)):
            node = self.points.iloc[idx]
            f.write(f"{int(node['Node Number'])} {node['X']:.10f} {node['Y']:.10f} {node['Z']:.10f}\n")
        f.write("\n")

        # Write elements
        f.write("ELEMENTS\n")
        for idx in range(len(self.elements)):
            elem = self.elements.iloc[idx]
            f.write(f"{int(elem['Element Number'])} {int(elem['Node1'])} {int(elem['Node2'])} ")
            f.write(f"{elem['Element Cross Section']:.10e} {elem['E']:.10e} {elem['V']:.10f}\n")
        f.write("\n")
```

Listing 7: E A DAT

5 M B Π Γ

H geometry_utils.py .

5.1 Π Γ Ά Y

```
def rotate_points_around_y(df: pd.DataFrame, angle_deg: float, origin=(0.0, 0.0, 0.0)) -> pd.DataFrame:
    """
    Rotate all points around the global Y-axis by angle_deg degrees.

    Args:
        df: DataFrame with columns ['X','Y','Z'] (and optionally 'Node Number']).
        angle_deg: Rotation angle in degrees (right-hand rule about +Y).
        origin: Pivot point (x0, y0, z0) to rotate about. Defaults to the global origin.

    Returns:
        A new DataFrame with rotated coordinates.
```

```
    """
    theta = np.deg2rad(angle_deg)
    c, s = np.cos(theta), np.sin(theta)
    # Rotation about Y: Ry = [[c,0,s],[0,1,0],[-s,0,c]]
    R = np.array([[c, 0.0, s],
                  [0.0, 1.0, 0.0],
                  [-s, 0.0, c]])

    coords = df[['X', 'Y', 'Z']].to_numpy(dtype=float)
    pivot = np.array(origin, dtype=float)
    rotated = (coords - pivot) @ R.T + pivot

    out = df.copy()
    out[['X', 'Y', 'Z']] = rotated
    return out
```

Listing 8: Σ Π Σ

O Y :

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix} \tag{9}$$

6 M E

O (solver.py) FEM.

6.1 Δ K

```
class FEMBeamSolver:
    """Handles FEM system assembly and solution for 3D beam elements"""

    def __init__(self):
        self.nodes = pd.DataFrame()
        self.elements = pd.DataFrame()
        self.boundary_conditions = {}
        self.loads = {}
        self.materials = {}

        # Beam parameters
        self.E = 0.0
        self.A = 0.0
        self.Iz = 0.0
        self.Iy = 0.0
        self.J = 0.0
        self.G = 0.0
        self.diameter = 0.0

        # Solution arrays (6 DOF per node)
        self.dof_per_node = 6
        self.K_global = None
        self.F_global = None
        self.u_global = None
        self.reactions = None
```

Listing 9: A K FEMBeamSolver

6.2 ϒ П M

O :

```python
def _compute_transformation_matrix(self, x1, y1, z1, x2, y2, z2):
    """Compute 3x3 transformation matrix for beam local coordinate system
    """
    # Element length and direction cosines
    dx = x2 - x1
    dy = y2 - y1
    dz = z2 - z1
    L = np.sqrt(dx**2 + dy**2 + dz**2)

    # Local x-axis (along beam element)
    x_local = np.array([dx/L, dy/L, dz/L])

    # Define auxiliary vector for determining local y and z axes
    # Use global Y-axis as reference, unless element is parallel to Y
    if abs(x_local[1]) > 0.9:  # Nearly vertical element
        aux = np.array([1.0, 0.0, 0.0])  # Use global X as auxiliary
    else:
        aux = np.array([0.0, 1.0, 0.0])  # Use global Y as auxiliary

    # Local z-axis (perpendicular to x_local and aux)
    z_local = np.cross(x_local, aux)
    z_local = z_local / np.linalg.norm(z_local)

    # Local y-axis (perpendicular to x and z)
    y_local = np.cross(z_local, x_local)
    y_local = y_local / np.linalg.norm(y_local)

    # 3x3 transformation matrix [x_local, y_local, z_local] as rows
    T = np.vstack([x_local, y_local, z_local])

    return T, L
```

Listing 10: ϒ П M

O 3×3 12×12 6 DOF :

```python
def _expand_transformation_matrix(self, T):
    """Expand 3x3 transformation to 12x12 for 6 DOF per node"""
    T_12x12 = np.zeros((12, 12))
    # Apply transformation to each 3x3 block (translations and rotations)
    for i in range(4):
        T_12x12[3*i:3*i+3, 3*i:3*i+3] = T
    return T_12x12
```

Listing 11: E П M

6.3 T П Δ

O , :

```python
def _create_local_stiffness_matrix(self, L):
    """Create 12x12 local stiffness matrix for 3D beam element"""
    k_local = np.zeros((12, 12))

    # Axial stiffness (DOF 0 and 6)
    EA_L = self.E * self.A / L
    k_local[0, 0] = EA_L
    k_local[0, 6] = -EA_L
    k_local[6, 0] = -EA_L
    k_local[6, 6] = EA_L

    # Torsional stiffness (DOF 3 and 9)
    GJ_L = self.G * self.J / L
    k_local[3, 3] = GJ_L
    k_local[3, 9] = -GJ_L
    k_local[9, 3] = -GJ_L
    k_local[9, 9] = GJ_L

    # Bending in local XY plane (about Z axis) - DOF 1, 5, 7, 11
    EIz_L3 = self.E * self.Iz / (L**3)
    k_local[1, 1] = 12 * EIz_L3
    k_local[1, 5] = 6 * EIz_L3 * L
    k_local[1, 7] = -12 * EIz_L3
    k_local[1, 11] = 6 * EIz_L3 * L

    k_local[5, 1] = 6 * EIz_L3 * L
    k_local[5, 5] = 4 * EIz_L3 * L**2
    k_local[5, 7] = -6 * EIz_L3 * L
    k_local[5, 11] = 2 * EIz_L3 * L**2

    # ... (similar terms for other DOF)

    return k_local
```

Listing 12: Σ T Π Δ (M)

6.4 Σ O Σ

O

:

```python
def assemble_stiffness_matrix(self):
    """Assemble global stiffness matrix for beam elements"""
    print("\nASSEMBLING GLOBAL STIFFNESS MATRIX (BEAM ELEMENTS)")
    print("="*80)

    num_nodes = len(self.nodes)
    n_dof = self.dof_per_node * num_nodes  # 6 DOF per node

    self.K_global = np.zeros((n_dof, n_dof))

    # Loop through all elements
    for idx in range(len(self.elements)):
        elem = self.elements.iloc[idx]

        node1_num = int(elem['Node1'])
```

```
        node2_num = int(elem['Node2'])

        # Get node coordinates
        node1 = self.nodes[self.nodes['Node Number'] == node1_num].iloc[0]
        node2 = self.nodes[self.nodes['Node Number'] == node2_num].iloc[0]

        x1, y1, z1 = node1['X'], node1['Y'], node1['Z']
        x2, y2, z2 = node2['X'], node2['Y'], node2['Z']
```

Listing 13: Σ O П Δ (M)

6.5 E Σ

M , NumPy:

```
def solve(self):
    """Solve the FEM system"""
    print("\nSOLVING FEM SYSTEM")
    print("="*80)

    # Identify fixed DOFs
    fixed_dofs = []
    for node_id, bc in self.boundary_conditions.items():
        node_idx = node_id - 1  # Convert to 0-based
        if bc['Ux'] == 1:
            fixed_dofs.append(node_idx * self.dof_per_node + 0)
        if bc['Uy'] == 1:
            fixed_dofs.append(node_idx * self.dof_per_node + 1)
        if bc['Uz'] == 1:
            fixed_dofs.append(node_idx * self.dof_per_node + 2)
        if bc['Rx'] == 1:
            fixed_dofs.append(node_idx * self.dof_per_node + 3)
        if bc['Ry'] == 1:
            fixed_dofs.append(node_idx * self.dof_per_node + 4)
        if bc['Rz'] == 1:
            fixed_dofs.append(node_idx * self.dof_per_node + 5)

    fixed_dofs = np.array(fixed_dofs, dtype=int)
    all_dofs = np.arange(len(self.F_global))
    free_dofs = np.setdiff1d(all_dofs, fixed_dofs)

    # Reduce system to free DOFs
    K_reduced = self.K_global[np.ix_(free_dofs, free_dofs)]
    F_reduced = self.F_global[free_dofs]

    # Check conditioning
    cond = np.linalg.cond(K_reduced)
    print(f"  Condition number: {cond:.2e}")

    if cond > 1e10:
        print("  WARNING: Stiffness matrix is ill-conditioned")

    # Solve for free DOFs
    u_reduced = np.linalg.solve(K_reduced, F_reduced)

    # Expand to full displacement vector
    self.u_global = np.zeros(len(self.F_global))
```

```
    self.u_global[free_dofs] = u_reduced

    print(f" System solved successfully")
```

Listing 14: E Σ

7 M M

O (postprocessor.py) .

7.1 Δ K

```
class FEMPostProcessor:
    """Handles visualization of FEM results"""

    def __init__(self):
        self.nodes = pd.DataFrame()
        self.elements = pd.DataFrame()
        self.displacements = pd.DataFrame()
        self.reactions = pd.DataFrame()
        self.element_forces = pd.DataFrame()
        self.summary = {}
```

Listing 15: A K FEMPostProcessor

7.2 O П K

O :

```
def plot_deformed_structure(self, magnification=1.0, show_plot=True):
    """Plot undeformed and deformed structure with interactive
   magnification slider"""
    print("\nCREATING DEFORMATION PLOT WITH INTERACTIVE SLIDER")
    print("="*80)

    fig = go.Figure()

    # Undeformed structure (always shown)
    edge_x_undef, edge_y_undef, edge_z_undef = [], [], []
    for idx in range(len(self.elements)):
        node1_num = int(self.elements.iloc[idx]['Node1'])
        node2_num = int(self.elements.iloc[idx]['Node2'])

        node1 = self.nodes[self.nodes['Node Number'] == node1_num].iloc[0]
        node2 = self.nodes[self.nodes['Node Number'] == node2_num].iloc[0]

        edge_x_undef.extend([node1['X'], node2['X'], None])
        edge_y_undef.extend([node1['Y'], node2['Y'], None])
        edge_z_undef.extend([node1['Z'], node2['Z'], None])

    fig.add_trace(go.Scatter3d(
        x=edge_x_undef, y=edge_y_undef, z=edge_z_undef,
        mode='lines',
        line=dict(color='cyan', width=4),
```

```
        name='Undeformed',
        visible=True,
        opacity=0.9
    ))

    # Create deformed structures for different magnifications
    magnifications = [1, 10, 50, 100, 500, 1000, 2000, 5000]

    for mag in magnifications:
        # Deformed structure
        deformed_nodes = self.nodes.copy()
        for idx in range(len(self.nodes)):
            node_num = int(self.nodes.iloc[idx]['Node Number'])
            disp = self.displacements[self.displacements['Node Number'] ==
   node_num].iloc[0]

            deformed_nodes.loc[idx, 'X'] += disp['Ux'] * mag
            deformed_nodes.loc[idx, 'Y'] += disp['Uy'] * mag
            deformed_nodes.loc[idx, 'Z'] += disp['Uz'] * mag
```

Listing 16: Г П K Slider (M)

7.3 O T П

O :

```
def plot_stress_distribution(self, show_plot=True):
    """Plot stress distribution on elements"""
    print("\nCREATING STRESS DISTRIBUTION PLOT")
    print("="*80)

    fig = go.Figure()

    # Get stress values
    stresses = self.element_forces['Stress'].values
    max_abs_stress = np.max(np.abs(stresses))

    # Color mapping
    edge_x, edge_y, edge_z, edge_colors = [], [], [], []

    for idx in range(len(self.elements)):
        node1_num = int(self.elements.iloc[idx]['Node1'])
        node2_num = int(self.elements.iloc[idx]['Node2'])

        node1 = self.nodes[self.nodes['Node Number'] == node1_num].iloc[0]
        node2 = self.nodes[self.nodes['Node Number'] == node2_num].iloc[0]

        stress = stresses[idx]

        # Create line segment for each element
        edge_x.extend([node1['X'], node2['X'], None])
        edge_y.extend([node1['Y'], node2['Y'], None])
        edge_z.extend([node1['Z'], node2['Z'], None])
        edge_colors.extend([stress, stress, None])
```

Listing 17: Г K T (M)

8 K Σ E

T FEM_Main.py :

```python
def main():
    """Execute the complete FEM analysis pipeline"""

    print("\n" + "="*80)
    print("FEM ANALYSIS - COMPLETE PIPELINE")
    print("="*80 + "\n")

    # Step 1: Pre-processor
    print("STEP 1: Running Pre-Processor...")
    print("-"*80)
    import preprocessor
    preprocessor.main()

    print("\n" + "="*80 + "\n")

    # Step 2: Solver
    print("STEP 2: Running Solver...")
    print("-"*80)
    import solver
    solver.main()

    print("\n" + "="*80 + "\n")

    # Step 3: Post-processor
    print("STEP 3: Running Post-Processor...")
    print("-"*80)
    import postprocessor
    postprocessor.main()

    print("\n" + "="*80)
    print("FEM ANALYSIS COMPLETED SUCCESSFULLY")
    print("="*80 + "\n")

    print("Results available in:")
    print("  - data/structure.dat (input data)")
    print("  - data/structure.res (results)")
    print("  - plots/ (interactive visualizations)")
    print("  - output_html/ (data tables)")
    print("\nOpen the HTML files in plots/ folder to view interactive
   results.")
```

Listing 18: Π Δ A FEM

9 M A Δ

9.1 M A E (.dat)

T :

```
PARAMETERS
E A Iz Iy J G diameter
```

```
MATERIALS
STEEL E nu

NODES
node_id x y z

ELEMENTS
elem_id node1 node2 cross_section E nu

BOUNDARY_CONDITIONS
node_id Ux Uy Uz Rx Ry Rz

LOADS
node_id Fx Fy Fz Mx My Mz

END
```

9.2 M A A (.res)

T :

```
DISPLACEMENTS
node_id Ux Uy Uz Rx Ry Rz |U| |R|

REACTIONS
node_id Rx Ry Rz Mx My Mz

ELEMENT_FORCES
elem_id Axial Shear_Y Shear_Z Torsion Moment_Y Moment_Z Stress Strain

SUMMARY
max_displacement value
max_stress value
max_strain value

END
```

10 Π X

Γ :

1. **O Γ** : E `_define_geometry_parameters()` `_add_initial_nodes()` `preprocessor.py`
2. **O O Σ** : Π `add_boundary_condition()`
3. **E Φ** : O `add_load()`
4. **E A** : E `python FEM_Main.py`
5. **Δ A** : A HTML `plots/`

11 Е

О . Г ,
`README.md`.

11.1 Δ П : А Т К

Г :

- 4
- 6
- 3 , 1

І Υ :

- $E = 210$ GPa (Х)
- $A = 4$ cm^2

Ф :

- $F_x = 1000$ N

О .

12 Σ

А FEM . Т :

- І 6 DOF
- А
- А
- О
- Е

12.1 М В

П :

- Υ
- М
- Δ
- А
- Δ GUI

13 А

1. Bathe, K. J. (1996). *Finite Element Procedures*. Prentice Hall.
2. Zienkiewicz, O. C., & Taylor, R. L. (2000). *The Finite Element Method*. Butterworth-Heinemann.
3. Cook, R. D., Malkus, D. S., Plesha, M. E., & Witt, R. J. (2001). *Concepts and Applications of Finite Element Analysis*. Wiley.

А П Λ К

Г , :

- `preprocessor.py` (752)
- `solver.py` (694)
- `postprocessor.py` (732)
- `geometry_utils.py` (29)
- `FEM_Main.py` (56)

В Е Λ

Н Python:

- `numpy`: А
- `pandas`: Δ
- `plotly`: Δ

Е :

```
pip install numpy pandas plotly
```